ECE 220 Computer Systems & Programming

Lecture 9 — Pointers and Arrays

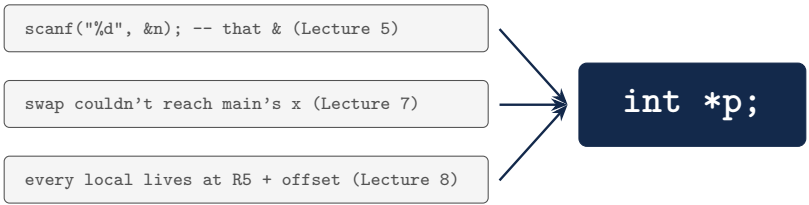
September 22, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Address Problem

You have been using addresses all along — today they get a name.



Today's Two Questions

A pointer is just an address — with a type.

1. What exactly does a pointer store — and what do `&` and `*` really do?
2. Why do arrays and pointers turn out to be (almost) the same thing?

Payoffs today: `swap` finally works, and functions get to modify whole arrays.

Declaring, &, and *

```c
/* & takes an address; * follows one */
#include <stdio.h>
int main() {
    int variable = 4;
    int *ptr;              /* declares a pointer to
     int */
    ptr = &variable;       /* ptr <- address of
     variable */
    *ptr = *ptr + 1;       /* changes... what? */
    printf("variable = %d\n", variable);
    printf("ptr      = %p\n", (void *)ptr);
    return 0;
}
```

- `&variable` — the **address** of `variable`
- `*ptr` — the **value at** the address in `ptr` (dereference)

In-class: at the end,

`variable` = _____

`ptr` = _____

The Pointer Toolkit

- **Declare**: `type *p1;` or `type* p2;` — identical (the `*` binds to the *name*)
- **Create**: `type var; type *vp = &var;`
- **Dereference**: `*vp`; twice for a pointer to a pointer: `**vpp`
- **NULL**: a pointer that points at nothing — `ptr = NULL;` dereferencing it:

- `void *`: an address with no type — must be converted before dereferencing

An uninitialized pointer points *somewhere* — writing through it is how programs corrupt themselves. Initialize to NULL or a real address.

In-Class Problem: swap, Fixed

Lecture 7's failure, resolved: pass *addresses*, not values.

- The parameters become `int *x, int *y`
- The call becomes _____
- Output now: _____

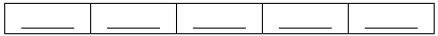
```c
/* swap, fixed: pass ADDRESSES, not values */
#include <stdio.h>

void swap(int *x, int *y) {
    /* declare a temporary */

    /* exchange the values x and y POINT TO */

}

int main() {
    int x = 2, y = 3;
    /* call swap with the ADDRESSES of x and y */

    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Declaring and Using Arrays

```
int grid[5] = {10, 11, 12, 13, 14};
int i;
grid[4] = grid[1] + 1;
for (i = 0; i < 2; i++)
    grid[i+1] = grid[i] + 2;
for (i = 0; i < 5; i++)
    printf("%d ", grid[i]);
printf("\navg = %d\n", average(grid, 5));
```

- Five `int`s, *contiguous* in memory; indices run 0. . . 4

In-class trace — final contents of `grid`:



Passing Arrays to Functions

```
int average(int array[], int size) {  /* == int *
     array */
    int i, sum = 0;
    for (i = 0; i < size; i++)
        sum = sum + array[i];
    return sum / size;
}
```

Equivalent parameter spellings:

`int array[10]` = `int array[]` = `int *array`

- Arrays are passed **by reference**: what goes into the activation record is _____
- So the callee can *modify* the caller's array — no copying of 10 (or 10 million) elements
- That is also why the *size* must be passed separately

Pointer–Array Duality

```
char word[4];  char *cptr;  cptr = word;
```

	pointer `cptr`	ptr notation `word`	array notation `word[]`
address of 1st element	`cptr`	`word`	_____
address of *n*th element	`(cptr+n)`	`(word+n)`	_____
value of 1st element	`*cptr`	`*word`	_____
value of *n*th element	`*(cptr+n)`	`*(word+n)`	_____

In-Class Problem: Reverse an Array

`array_reversal(a, n)`: first element becomes last, second becomes second-to-last, ...

- How many swaps for n elements?

  _____

- `array[i]`'s mirror is

  _____

- Why does `main` see the change?

```
void array_reversal(int array[], int n) {
    /* declare a loop counter and a temporary */

    /* how far does the loop need to go? */

        /* swap array[i] with its mirror element */

}
```

Input `1 2 3 4 5 6` → output _____

Summary & What's Next

Today

- **Pointers**: an address with a type; `&` takes an address, `*` follows one; NULL and uninitialized pointers are the sharp edges
- **swap works** when you pass addresses — still pass-by-value, but the value is an address
- **Arrays**: contiguous, 0-indexed, passed by reference (just the address of element 0) — and `a[n]` is `*(a+n)`

Next lecture

- Strings (arrays of char with a terminator) and multi-dimensional arrays

Code from today: `pointers.c`, `swap_ptr.c`, `arrays.c`, `reverse.c`.